

Taller: Apuntadores

Duración Estimada: 1:20

Objetivo

Comprender el funcionamiento básico de los apuntadores en C++ mediante ejercicios prácticos relacionados con direcciones de memoria, desreferenciación y modificación de variables. Asimismo, se busca fortalecer la capacidad del estudiante para resolver problemas de programación bajo un límite de tiempo, con el propósito de desarrollar un pensamiento ágil y evitar bloqueos ante situaciones de presión.

Para ello, se utilizará un cronómetro durante los ejercicios, motivando al estudiante a resolver cada problema en el menor tiempo posible. Sin embargo, completar el ejercicio dentro del tiempo establecido no será el objetivo principal, la actividad busca principalmente desarrollar la capacidad de mantener la calma, analizar el problema y aplicar los conocimientos adquiridos incluso cuando existe la presión del tiempo.

Todas las respuestas se encontrarán en el archivo: `solucion-taller-apuntadores.cpp`

Conceptos para el taller:

Antes de comenzar los ejercicios, recuerde:

- `&` permite obtener la dirección de memoria de una variable.
- `*` permite acceder al valor almacenado en la dirección que contiene un apuntador.
- Un apuntador almacena una dirección de memoria.

Ejemplo:

```
int número = 10;
```

```
int* puntero = &número;
```

En este caso:

- `numero` almacena 10.
- `&número` representa la dirección de `numero`.
- `puntero` almacena la dirección de `numero`.

Ejemplo: `(0xbb2ffac4)`

- `*puntero` permite acceder al valor de `numero`. (desreferenciación)

Ejemplo: `(10)`

Parte 1. Conceptos básicos de apuntadores

Cree un programa que permita explorar la relación entre una variable, su dirección de memoria y un apuntador.

Instrucciones

1. Declare una variable entera llamada número con el valor 25.
2. Declare un apuntador llamado apuntador_25 que almacene la dirección de numero.
3. Imprima por consola:
 - El valor de numero.
 - La dirección de memoria de numero.
 - El valor obtenido mediante la desreferenciación de apuntador_25.

Pregunta

¿Qué relación existe entre &numero y el valor almacenado en apuntador_25?

- A) &numero y apuntador_25 representan la dirección de memoria de numero.
- B) &numero representa el valor de numero y apuntador_25 representa su dirección.
- C) &numero representa la dirección de apuntador_25 y apuntador_25 representa el valor de numero.
- D) &numero y apuntador_25 representan el valor almacenado en numero.

Parte 2 . Modificación de variables mediante apuntadores

En esta parte se utilizará un apuntador para modificar directamente el valor de una variable.

Implemente la siguiente función:

```
void duplicar (int* numero);
```

La función debe recibir la dirección de un número entero y duplicar el valor de la variable utilizando el apuntador.

Ejemplo

```
int numero = 8;
```

```
duplicar(int * apuntador_10);
```

```
cout << numero;
```

El resultado esperado es:

16

Restricciones

- La función duplicar debe recibir un apuntador como parámetro.
- Dentro de la función debe utilizar la **desreferenciación** (*) para modificar el valor.
- No debe utilizar return para devolver el nuevo valor.
- La variable original debe quedar modificada después de llamar a la función.

Pregunta

¿Por qué al modificar *número dentro de la función también se modifica la variable original?

- A) Porque el apuntador contiene una copia independiente de la variable.
- B) Porque el apuntador contiene la dirección de memoria de la variable y la desreferenciación permite modificar el valor almacenado allí.
- C) Porque todas las variables enteras se modifican automáticamente al pasar por una función.
- D) Porque return modifica automáticamente la variable original.

Parte 3 .Conteo de números pares mediante apuntadores

Cree una función que reciba un arreglo de números enteros mediante un apuntador y determine cuántos de sus elementos son números pares.

La función debe tener la siguiente estructura:

```
int contarPares(int* arreglo, int tamanio);
```

Ejemplo:

Dado el siguiente arreglo:

```
int numeros[6] = {4, 7, 10, 13, 20, 25};
```

Al ejecutar:

```
cout << contarPares(numeros, 6);
```

El resultado esperado es:

3

Los números pares encontrados son:

4, 10, 20

Restricciones

- La función debe recibir el arreglo mediante un apuntador.
- Debe utilizar la desreferenciación (*) para acceder a los elementos.
- Debe utilizar aritmética de apuntadores para recorrer el arreglo.
- No puede utilizar la expresión arreglo[i].
- La función debe retornar la cantidad de números pares encontrados.

Pregunta

¿Qué permite hacer la expresión *(arreglo + i)?

- A) Obtener la dirección del elemento ubicado en la posición i.
- B) Obtener el valor del elemento ubicado en la posición i.
- C) Modificar automáticamente todos los elementos del arreglo.

D) Obtener el tamaño total del arreglo.

Parte 4. Invertir un arreglo mediante apuntadores

El estudiante debe crear una función que **invierta los elementos de un arreglo** utilizando apuntadores.

Por ejemplo:

`int numeros [5] = {5, 4, 7, 11, 12};`

Después de ejecutar la función:

50 40 30 20 10

Función:

`void invertir (int* arreglo, int tamano);`

Restricciones:

- Debe utilizar apuntadores para acceder a los elementos.
- No puede utilizar `arreglo[i]`.
- No puede crear un segundo arreglo para almacenar el resultado.
- Debe modificar el arreglo original.

Pregunta:

¿Si arreglo apunta al primer elemento y $i = 3$ ¿qué representa $*(arreglo + i)$?

- A) La dirección del cuarto elemento.
- B) El tamaño del cuarto elemento.
- C) La dirección del primer elemento.

D) El valor almacenado en el cuarto elemento

Parte 5. Memoria dinámica con new y delete[]

Cree un programa que permita al usuario ingresar la cantidad de números que desea almacenar y posteriormente calcule cuántos de ellos son positivos.

Instrucciones

1. Solicite al usuario el tamaño del arreglo.

2. Reserve dinámicamente la memoria utilizando new.
3. Solicite al usuario los valores del arreglo.
4. Utilice un apuntador para recorrer el arreglo.
5. Cuente cuántos números son positivos.
6. Muestre la cantidad de números positivos.
7. Libere la memoria utilizando delete[].

Ejemplo

Si el usuario ingresa:

Ingrese el tamaño: 6

Ingrese los valores:-4,8,-2,15,0,7

El programa debe mostrar:

Cantidad de números positivos: 3

Los números positivos son:

8, 15, 7

Restricciones

- Debe utilizar new para reservar el arreglo dinámicamente.
- Debe utilizar delete[] para liberar la memoria.
- Debe utilizar un apuntador para acceder a los elementos.
- No puede utilizar vector.

Parte 6. Análisis general:

Analice el siguiente fragmento de código:

```

void analizar() {
    cout<<"-----6 parte -----"<<endl;
    int numero = 10;
    int* puntero = &numero;
    *puntero = 20; int numeros[3]= {5, 10, 15};
    int* arreglo = numeros;
    *(arreglo + 1) = *puntero;
    int* dinamico = new int;
    *dinamico = *(arreglo + 2) + *puntero;
    cout << numero << endl; cout << numeros[1] << endl;
    cout << *dinamico << endl; delete dinamico;
}

```

Pregunta 1

¿Qué valor tendrá numero cuando se ejecute:

cout << numero << endl;

- A) 10
- B) 15
- C) 20
- D) 25

Pregunta 2

¿Qué valor tendrá numeros[1] después de ejecutar:

*(arreglo + 1) = *puntero;

- A) 5
- B) 10
- C) 15
- D) 20

Pregunta 3

¿Qué valor se almacenará en la memoria reservada por:

int* dinamico = new int;

después de ejecutar:

*dinamico = *(arreglo + 2) + *puntero;

- A) 20
- B) 25
- C) 30
- D) 35

Pregunta 4

¿Qué función cumple:

delete dinamico;

- A) Elimina el valor de dinamico, pero mantiene la memoria reservada.
- B) Libera la memoria que había sido reservada mediante new.
- C) Elimina el arreglo numeros.
- D) Cambia dinamico a nullptr.